

Bitte beantworten Sie die Fragen in den dafür vorgesehenen Feldern. Wenn Sie keinen Platz mehr für eine Antwort haben, fahren Sie bitte auf der Rückseite fort und geben an wo es weiter geht.
Abgesehen von Internetfähigen Geräten sind alle Hilfsmittel zugelassen.
Schreiben Sie Ihre Klausur bitte nicht mit Bleistift und lassen Sie die Klausur bitte geheftet.
Sie haben 90 Minuten Zeit. Mit 43 Punkten gilt die Klausur als bestanden.

Vor- und Nachname (in Druckbuchstaben): _____

Matrikelnummer und Studiengang: _____

Aufgabe Nr.:	Punktzahl:	Davon erreicht:
1	5	
2	10	
3	6	
4	12	
5	8	
6	6	
7	7	
8	7	
9	7	
10	9	
11	13	
Summe	90	

Note und Unterschrift des Erstprüfers: _____

Unterschrift des Zweitprüfers: _____

Lernraum I

1. (5 Punkte) Geben Sie die Laufzeit für den folgenden Code in der Θ -Notation an. Es sollte ersichtlich sein wie Sie auf diese Laufzeit gekommen sind (bspw. sehr kurze Begründung oder Gleichung).

```
n = array.length()
i = n
while (i >= 2):
    if i <= n/2:
        i--
    else:
        i = i/2
```

.....

.....

.....

.....

.....

.....

.....

2. (10 Punkte) Schauen Sie sich folgenden Python Code an, in dem in einem Dictionary (d.h. einer Hashtabelle) Studierende gespeichert werden.

```
class Studierende:
    def __init__(self):
        self.students = {}

    def add_student(self, matrikelnummer, student):
        first_digit = matrikelnummer[0]
        if first_digit not in self.students:
            self.students[first_digit] = []
        self.students[first_digit].append((matrikelnummer, student))

    def search_student(self, matrikelnummer):
        first_digit = matrikelnummer[0]
        if first_digit in self.students:
            for m_num, student in self.students[first_digit]:
                if m_num == matrikelnummer:
                    return student
        return None

# Beispielverwendung
studierende = Studierende()

studierende.add_student("11101234", "Max Mustermann")
```

Stellen Sie sich vor, dass `add_student` n -mal aufgerufen wurde und danach n Studierende in dem Dictionary gespeichert sind.

Welche Laufzeit, in der Θ -Notation, wird die Methode `search_student` im Best Case und im Worst Case haben, wenn n Studierende in dem Dictionary gespeichert sind. Wann tritt der Best Case und wann der Worst Case ein? Sie können hier mit $n = 10$ arbeiten.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Lernraum II

3. (6 Punkte) Eine Textverarbeitungssoftware möchte die Häufigkeit jedes Wortes in einem Dokument zählen. Welche Datenstruktur würden Sie verwenden und warum? Skizzieren Sie einen Algorithmus in Pseudocode oder Wörtern zur Implementierung dieser Aufgabe. Der Algorithmus sollte eine lineare Laufzeit haben. n entspricht der Anzahl der Wörter im Dokument.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

4. (12 Punkte) Ein soziales Netzwerk möchte Empfehlungen für neue Freunde basierend auf gemeinsamen Freunden geben. Entwickeln Sie einen Algorithmus in Pseudocode oder Wörtern, der einem Nutzer andere Nutzer zur Freundschaft vorschlägt. Dabei sollen solche Nutzer vorgeschlagen werden, die mit möglichst vielen Freunden des ursprünglichen Nutzers befreundet sind. In welcher Datenstruktur würden Sie die Anzahl gemeinsamer Freunde pro durchsuchten Nutzer speichern? Folgender Graph dient als Hilfestellung.

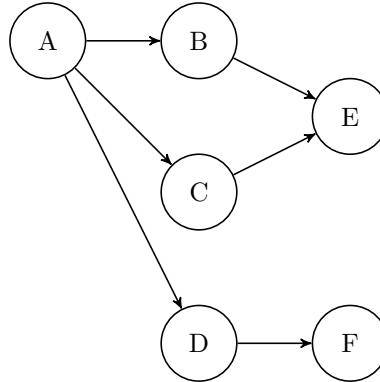


Abbildung 1: Der Knoten A ist der Nutzer dem neue Freunde vorgeschlagen werden sollen. Knoten B, C und D sind bereits Freunde von Knoten A. Der Algorithmus sollte Knoten E dem Nutzer A als neuen Freund vorschlagen, da E mit zwei Freunden von Knoten A befreundet ist. Knoten F soll Knoten A erst vorgeschlagen werden, wenn Knoten E bereits Knoten A vorgeschlagen wurde.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

5. (8 Punkte) Eine Bibliothek verwaltet ihre Bücher in einem AVL-Baum, um schnelle Suchoperationen zu ermöglichen. Und zwar sollen bei der Suche nach einem Autor, bspw. J.K. Rowling, alle Bücher des Autors möglichst schnell angezeigt werden. Was wäre ein geeigneter **eindeutiger** Schlüssel und was ein geeigneter Wert für jeden Knoten im AVL-Baum, die diese Eigenschaft erfüllen? Was müsste verändert werden, wenn zusätzlich auch schnell nach der ISBN-Nummer gesucht werden können soll? Zur Beantwortung der zweiten Frage dürfen Sie auch andere oder weitere Datenstrukturen nutzen.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Lernraum III

6. (6 Punkte) Ein Navigationssystem möchte die kürzeste Route zwischen zwei Standorten berechnen. Welchen Algorithmus und welche Datenstrukturen in dem Algorithmus würden Sie dafür verwenden?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

7. Modellieren Sie ein Rezept zum Kuchen backen als Graph und geben Sie die Mindestdauer für den Gesamtvorgang an unter der Annahme, dass parallel durchführbare Vorgänge die Dauer der einzelnen Vorgänge nicht verlängert. Das Rezept hat die folgenden 9 Schritte und die Abhängigkeiten sind wie folgt gegeben.

1. Zutaten abwiegen: Muss als erstes gemacht werden, keine Abhängigkeiten. Dauer: 5 Minuten.
2. Zutaten mischen: Abhängig von "Zutaten abwiegen". Dauer: 5 Minuten
3. Teig kneten: Abhängig von "Zutaten mischen". Dauer: 10 Minuten
4. Backform vorbereiten: Kann unabhängig von den ersten Schritten durchgeführt werden. Dauer: 1 Minute
5. Teig in die Backform füllen: Abhängig von "Teig kneten" und "Backform vorbereiten". Dauer: 1 Minute
6. Ofen vorheizen: Kann parallel zu anderen Schritten erfolgen, aber muss vor dem Backen abgeschlossen sein. Dauer: 15 Minuten
7. Kuchen backen: Abhängig von "Teig in die Backform füllen" und "Ofen vorheizen". Dauer: 20 Minuten
8. Kuchen abkühlen lassen: Abhängig von "Kuchen backen". Dauer: 30 Minuten.
9. Kuchen verzieren: Abhängig von "Kuchen abkühlen lassen". Dauer: 8 Minuten.

(a) (6 Punkte) Graph:

[illegible]

(b) (1 Punkt) Gesamtdauer des Vorgangs:

.....

8. Angenommen, Sie sind Manager:in eines Telekommunikationsunternehmens und haben die Aufgabe, Glasfaserkabel zu verlegen, um mehrere Städte miteinander zu verbinden. Die Kosten für das Verlegen von Glasfaserkabeln zwischen den Städten sind unterschiedlich, und Ihr Ziel ist es, eine kostengünstige Netzwerkinfrastruktur zu schaffen.

- (a) (3 Punkte) Welche Datenstruktur wählen Sie um das Problem zu modellieren? Wie modellieren Sie Städte, Glasfaserkabel und die Kosten für das Verlegen der Glasfaserkabel?

.....

.....

.....

.....

.....

.....

- (b) (1 Punkt) Mit welchem Algorithmus bestimmen Sie die kostengünstigste Möglichkeit, die Städte über Glasfaser miteinander zu verbinden?

.....

.....

.....

.....

.....

.....

- (c) (3 Punkte) Welchen Vor- und welchen Nachteil hat die Lösung, die durch den Algorithmus in (b) bestimmt wird?

.....

.....

.....

.....

.....

.....

9. (7 Punkte) Weighted Quick-Union mit Pfadkomprimierung: Rufen Sie `union(9, 24)` auf die folgenden beiden Bäume auf. Nutzen Sie dabei Weighted Quick-Union mit Pfadkomprimierung. Zeichnen Sie den sich ergebenden Baum. Hinweis: Es dürfte hilfreich sein Zwischenschritte zu zeichnen.

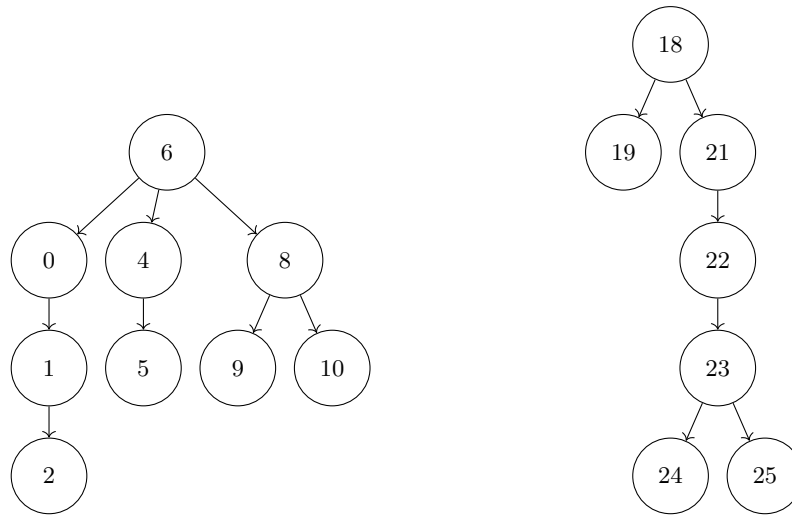
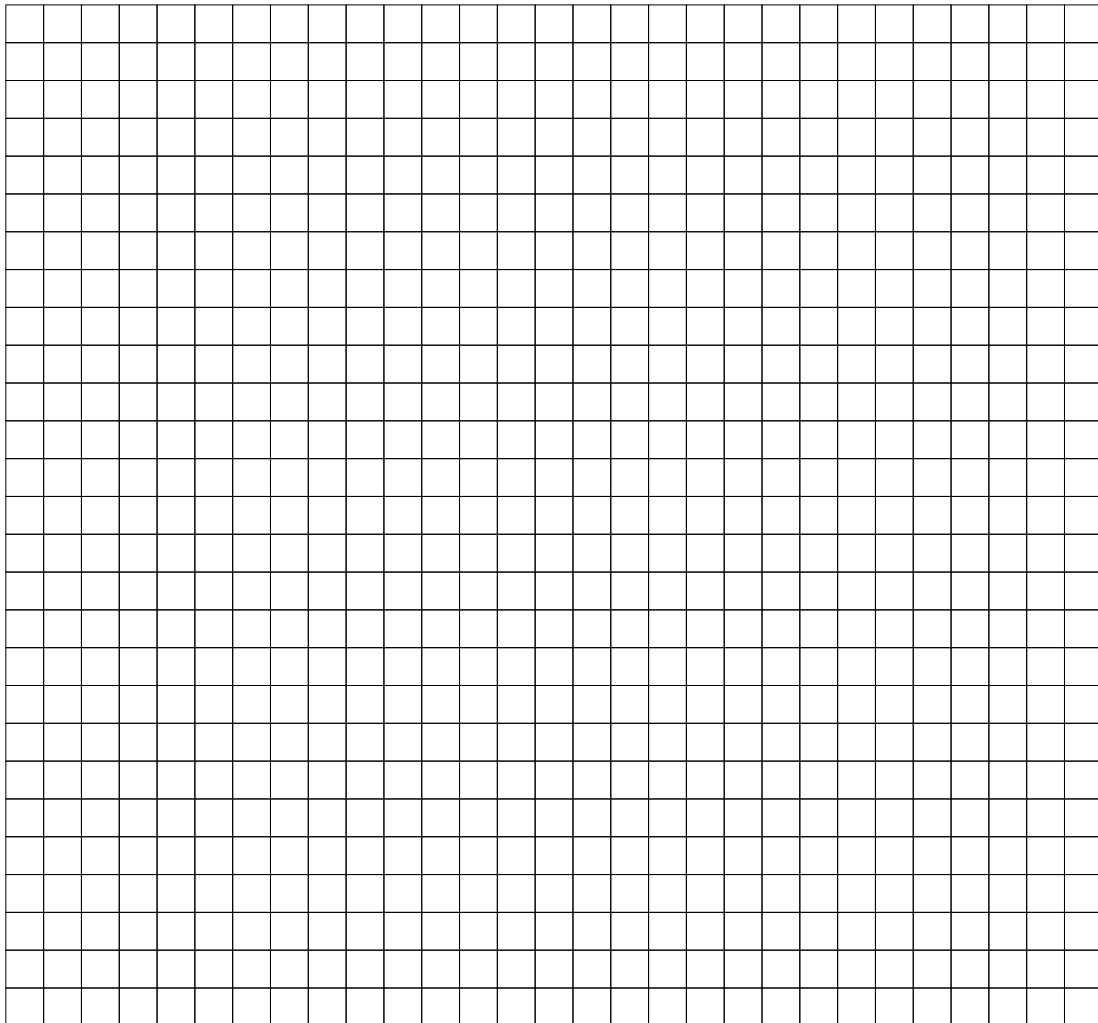


Abbildung 2: Darstellung von zwei Disjunkten Mengen auf denen Sie `union` aufrufen sollen.



Lernraum IV

10. Wählen Sie für jedes der folgenden Szenarien den passenden Sortieralgorithmus und begründen Sie Ihre Wahl in einem kurzen Satz.

Zur Wahl stehen: Insertionsort, Selectionsort, Mergesort, Quicksort, Heapsort, Bucketsort

- (a) (3 Punkte) Sie schreiben ein Programm, das die Bewerber für ein prestigeträchtiges Stipendium sortiert. Jeder Bewerbung wird eine numerische Punktzahl zugewiesen, um eine geordnete Rangfolge der Bewerbungen zu erstellen. Wenn zwei Bewerbungen die gleiche Punktzahl haben, wird die Bewerbung ausgewählt, die zuerst eingegangen ist.

.....

.....

.....

.....

.....

- (b) (3 Punkte) Ein Softwareunternehmen möchte die Performance seiner Mitarbeiter für ein bestimmtes Projekt evaluieren. Die Performance-Werte der Mitarbeiter liegen zwischen 0 und 10 (Ganzzahlen), wobei es sehr viele Mitarbeiter gibt. Sie möchten die Performance-Werte effizient sortieren, um die besten und schlechtesten Leistungen leicht identifizieren zu können.

.....

.....

.....

.....

.....

- (c) (3 Punkte) Ein Personalvermittler sortiert Bewerber nach ihren Ergebnissen in Eignungstests. Die Ergebnisse müssen häufig aktualisiert werden, wenn neue Bewerbungen eingehen.

.....

.....

.....

.....

.....

11. (13 Punkte) Das Partitionsproblem besteht darin, zu bestimmen, ob eine gegebene Menge von Ganzzahlen in zwei Teilmengen aufgeteilt werden kann, so dass die Summe der Elemente in beiden Teilmengen gleich ist.

Bspw. kann die Liste [3, 4, 1] in [3, 1] und [4] partitioniert werden, da die Summe der Elemente in beiden Listen 4 beträgt.

Die Brute-Force Lösung lautet:

```
def isSubsetSum(arr, n, sum):
    if sum == 0:
        return True
    if n == 0 and sum != 0:
        return False

    if arr[n-1] > sum:
        return isSubsetSum(arr, n-1, sum)

    return isSubsetSum(arr, n-1, sum) or isSubsetSum(arr, n-1, sum-arr[n-1])

def findPartion(arr, n):
    sum = 0
    for i in range(0, n):
        sum += arr[i]

    if sum % 2 != 0:
        return false

    return isSubsetSum(arr, n, sum // 2)

if __name__ == '__main__':
    arr = [3, 4, 1]
    n = len(arr)

    if findPartion(arr, n) == True:
        print("Can be divided into two subsets of equal sum")
    else:
        print("Can not be divided into two subsets of equal sum")
```

Zeichnen Sie dazu den Rekursionsbaum der Methode `isSubsetSum` für das im Code aufgerufene Beispiel. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `n` und `sum` einzuzeichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:



Rechenkästchen folgen auf der nächsten Seite.

